

8. Pengolahan String dan Regular Expression

8.1 Tujuan Praktikum

Setelah mempelajari Bab ini, mahasiswa diharapkan dapat:

1. Dapat menjelaskan tentang String.
2. Dapat mengakses String dan memanipulasi String.
3. Dapat menggunakan operator metode String.
4. Dapat melakukan parsing String sederhana.
5. Menggunakan *regular expression*

8.2 Alat dan Bahan

Praktikum ini membutuhkan perangkat komputer yang memiliki spesifikasi minimum sebagai berikut:

1. Terkoneksi ke Internet dan dapat mengunduh package-package Python.
2. Mampu menjalankan sistem operasi Windows 10 atau Ubuntu Linux.

Perangkat lunak yang diperlukan untuk mendukung praktikum ini adalah sebagai berikut:

1. Python 3.7 atau 3.8 yang terinstall menggunakan Anaconda atau Installer Python lainnya.
2. Web Browser (Mozilla Firefox, Microsoft Edge atau Google Chrome).
3. Command Prompt (jika menggunakan Windows).
4. Terminal (jika menggunakan Linux).
5. Editor Python (Visual Studio Code, PyCharm, Spyder atau editor-editor lainnya yang mendukung Python).

8.3 Materi

8.3.1 Pengantar String

String adalah untaian karakter-karakter yang menjadi satu kesatuan dan digunakan dalam program komputer untuk menyimpan kalimat, baik panjang ataupun pendek. String adalah suatu jenis data yang mampu menyimpan huruf / karakter dan disimpan dalam kode ASCII. Tidak semua

bahasa pemrograman memiliki tipe data String, seperti misalnya bahasa C. Tipe data ini merupakan jenis tipe data yang bukan tipe data dasar, karena tipe data ini pada dasarnya menyimpan lebih dari satu nilai tunggal sebagai satu kesatuan. Beberapa bahasa pemrograman ada yang menyebutkan bahwa String pada dasarnya adalah kumpulan tipe data karakter / array of character / list of character.

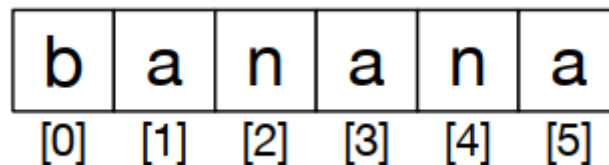
8.3.2 Pengaksesan String dan Manipulasi String

String dapat dibuat dengan sederhana menggunakan variabel:

```
1   namasaya = "Antonius Rachmat C"
2   temansaya1 = "Yuan Lukito"
3   temansaya2 = 'Laurentius Kuncoro'
4   temansaya3 = "Matahari" + 'Bakti'
5
6   print(temansaya3)
7   print(namasaya[0]) #'A'
8   print(namasaya[9]) #'R'
9   print(temansaya1[1]) #'u'
10
11  huruf = temansaya2[0]
12  print(huruf) #'L'
```

Jadi string pertama kali dibuat dengan deklarasi variabel dan langsung diisi dengan data, string dapat diakses sebagai satu kesatuan dengan menyebut nama variabelnya, atau per huruf dengan menyebutkan indeksinya. Indeks pada string dimulai dari 0 seperti layaknya pada list. Indeks string haruslah berupa bilangan bulat, bukan pecahan.

Pada memory komputer, string disimpan secara urut menggunakan list yang berisi huruf-huruf dengan indeks yang dimulai dari nol. Gambar 8.1 menunjukkan string berisi "banana" yang disimpan pada memory komputer.



Gambar 8.1: Bentuk string di dalam memory

8.3.3 Operator dan Metode String

OPERATOR in

Pada String kita dapat memeriksa apakah suatu kalimat merupakan substring dari suatu kalimat lain dengan menggunakan operator in. Hasil dari operator ini adalah True / False.

```
1   kalimat = "saya mau makan"
2   data = "saya"
3   print(data in kalimat) #True
4   print("mau" in kalimat) #True
```

```
5     print("dia" in kalimat) #False
```

Selain operator in, pada String juga dapat dilakukan perbandingan (comparison) yang juga menghasilkan True atau False.

```
1     if "saya" > "dia":
2         print("Ya") #Ya
3     else:
4         print("Tidak")
5
6     if "dua" == "dua":
7         print("Sama") #Sama
```

FUNGSI len

Cara untuk mengetahui berapa panjang (berapa jumlah karakter) dari sebuah string adalah dengan menggunakan operator len(<string>). Untuk menampilkan huruf terakhir dari sebuah string kita harus menggunakan indeks string yang ke- len(<string>-1), sebab indeks dimulai dari 0.

Contoh program Python adalah:

```
1     kalimat = "universitas kristen duta wacana yogyakarta"
2     print(len(kalimat)) #output 42
3
4     terakhir = kalimat[len(kalimat)-1]
5     print(terakhir) #output 'a'
6
7     #bisa juga menggunakan indeks -1
8     terakhir_versi2 = kalimat[-1]
9     print(terakhir_versi2) #output 'a'
10    #atau menggunakan indeks -2 untuk huruf terakhir kedua
11    terakhir2 = kalimat[-2]
12    print(terakhir2) #output 't'
```

TRAVERSING STRING

Untuk dapat menampilkan string dengan cara ditampilkan huruf demi huruf adalah dengan menggunakan loop yang dilakukan per huruf dengan 2 cara:

- Dilakukan dengan akses terhadap indeks

```
1     kalimat = "indonesia jaya"
2     i = 0
3     while i < len(kalimat):
4         print(kalimat[i], end=' ')
5         i += 1
```

- Dilakukan tanpa akses terhadap indeks secara otomatis

```
1     kalimat = "indonesia jaya"
2     for kal in kalimat:
3         print(kal, end=' ')
```

STRING SLICE

String slice adalah menampilkan substring pada sebuah string dengan menggunakan indeks dari awal tertentu sampai akhir-1 tertentu. Sintaksnya menggunakan `<string>[awal:akhir]`. Bagian awal atau akhir boleh dikosongkan. Bagian awal dimulai dari 0.

```
1 kalimat = "cerita rakyat"
2 awal = 0
3 akhir = 6
4 print(kalimat[awal:akhir]) #cerita
5 print(kalimat[7:len(kalimat)]) #rakyat
6 print(kalimat[:5]) #cerit
7 print(kalimat[5:]) #a rakyat
8 print(kalimat[:]) #cerita rakyat
```

String merupakan data yang bersifat immutable! Immutable adalah bahwa data tersebut tidak bisa diubah saat program berjalan, hanya bisa diinisialisasi saja. Contoh:

```
1 kalimat = "satu"
2 kalimat[0] = "batu" #TypeError: 'str' object does not support item assignment
```

Agar bisa diubah, maka harus disimpan dalam variabel yang berbeda.

```
1 kalimat = "satu"
2 kalimat_baru = kalimat[0] + "alah" #salah
```

Berikut adalah beberapa method String yang sering digunakan:

Nama Method	Kegunaan	Penggunaan
<code>capitalize()</code>	untuk mengubah string menjadi huruf besar	<code>string.capitalize()</code>
<code>count()</code>	menghitung jumlah substring yang muncul dari sebuah string	<code>string.count()</code>
<code>endswith()</code>	mengetahui apakah suatu string diakhiri dengan string yang diinputkan	<code>string.endswith()</code>
<code>startswith()</code>	mengetahui apakah suatu string diawali dengan string yang diinputkan	<code>string.startswith()</code>
<code>find()</code>	mengembalikan indeks pertama string jika ditemukan string yang dicari	<code>string.find()</code>
<code>islower()</code> dan <code>isupper()</code>	mengembalikan True jika string adalah huruf kecil / huruf besar	<code>string.islower()</code> dan <code>string.isupper()</code>
<code>isdigit()</code>	mengembalikan True jika string adalah digit (angka)	<code>string.isdigit()</code>
<code>strip()</code>	menghapus semua whitespace yang ada di depan dan di akhir string	<code>string.strip()</code>
<code>split()</code>	memecah string menjadi token-token berdasarkan pemisah, misalnya berdasarkan spasi	<code>string.split()</code>

Dan masih banyak lainnya. Ingat semua fungsi/method di atas yang mengembalikan string, mengembalikan string baru, tidak mengubah yang aslinya, karena string bersifat *immutable*. Daftar lengkap dapat dilihat di <https://docs.python.org/library/stdtypes.html#string-methods>

Operator * dan + pada String

Pada Python operator + dan * memiliki kemampuan khusus. Operator + yang biasanya digunakan untuk menjumlahkan bilangan bisa digunakan untuk menggabungkan dua buah string. Sedangkan operator * yang bisa digunakan untuk mengkalikan bilangan bisa digunakan untuk menampilkan string sejumlah perkaliannya. Perhatikan kode berikut:

```
1 kata1 = "saya"
2 kata2 = "makan"
3 kata3 = kata1 + " " + kata2
4 print(kata3)      #hasil adalah penggabungan: saya makan
5 kata4 = "ulang"
6 print(kata4 * 4)   #hasil adalah ulangulangulangulang
7 kata4 = "ulang "
8 print(kata4 * 2)   #hasil adalah ulang ulang
```

8.3.4 Parsing String

Parsing string adalah cara menelusuri string bagian demi bagian untuk mendapatkan / menemukan / mengubah bagian string yang diinginkan. Perhatikan contoh berikut:

"Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka".

Dilihat dari string di atas, diinginkan untuk mengambil tanggal bulan tahun dari kalimat di atas dan disusun ulang menjadi format 08/17/1945. Kita dapat menggunakan cara parsing string misalnya:

- Spit string berdasarkan spasi sehingga menjadi token: "Saudara-saudara", "pada", "tanggal", "17-08-1945", "Indonesia", dan "merdeka".
- Dari sini, lanjutkan dengan mencari token yang diawali dengan angka, kemudian lanjutkan dengan split angka tersebut dengan pemisah '-'.
- Kemudian susun ulang token-token dari langkah sebelumnya sehingga berbentuk seperti yang diinginkan.

Berikut adalah contoh penyelesaiannya:

```
1 kalimat = "Saudara-saudara, pada tanggal 17-08-1945 Indonesia merdeka"
2
3 hasil = kalimat.split(" ")
4
5 for kal in hasil:
6     if kal[0].isdigit():
7         hasil2 = kal.split("-")
8         print(hasil2[1]+"/"+hasil2[0]+"/"+hasil2[2])
```

Hasil dari keluaran program adalah **08/17/1945**

8.3.5 Pengantar Regex

Pada bab String, kita sudah sedikit mempelajari mengenai teknik-teknik pengaksesan string, manipulasi string, dan berbagai kasus-kasus pengolahan string lainnya, termasuk string yang terdapat pada file. Dari pengalaman tersebut dapat dilihat bahwa kita cukup kesulitan untuk melakukan pengolahan string dengan teknik biasa / standar. Terdapat teknik pengolahan string yang lebih mudah dan cepat dengan menggunakan bantuan regular expression.

Regular expression adalah ekspresi pola yang berbentuk kumpulan karakter yang digunakan untuk menemukan pola (pattern) yang sama dengan pola regex di dalam string lain yang ingin dicari. Regex membantu kita dalam pencarian string dengan pola tertentu, mengganti string dengan pola tertentu, dan menghapus string dengan pola tertentu. Intinya regex membantu dalam parsing string yang selama ini biasanya hanya menggunakan perintah `split()` dan `find()` saja.

Regex sangat powerful dalam searching dan extracting pola namun memiliki pola yang cukup rumit. Tidak semua bahasa pemrograman mendukung *regular expression library*. Python merupakan salah satu bahasa yang mendukung library regex dengan cara `import re`. Salah satu fungsi yang paling mudah digunakan dari library `re` adalah `search()`.

Dengan menggunakan file `mbox-short.txt`, kita akan mencoba menampilkan semua string pada file tersebut yang **mengandung** pola "From: ".

```
1     import re
2     handle=open('mbox-short.txt')
3     count = 0
4     for line in handle:
5         line=line.rstrip()
6         if re.search('From:', line):
7             count += 1
8         print(line)
9         print("Count: ",count)
```

Dari kode di atas kita dapat melihat bahwa `re.search` bisa saja diganti dengan menggunakan perintah **find()** pada string biasa. Pola pada contoh di atas belum menggunakan kemampuan regex yang seutuhnya.

```
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: zqian@umich.edu
From: rjlowe@iupui.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
From: gsilver@umich.edu
From: gsilver@umich.edu
From: zqian@umich.edu
From: gsilver@umich.edu
From: wagnermr@iupui.edu
From: zqian@umich.edu
From: antranig@caret.cam.ac.uk
From: gopal.ramasammycook@gmail.com
From: david.horwitz@uct.ac.za
From: david.horwitz@uct.ac.za
From: david.horwitz@uct.ac.za
From: david.horwitz@uct.ac.za
From: stephen.marquard@uct.ac.za
From: louis@media.berkeley.edu
From: louis@media.berkeley.edu
```

```

From: ray@media.berkeley.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
From: cwen@iupui.edu
Jumlah: 27

```

Jika diinginkan mencari baris yang **diawali** dengan pola "From", maka kita harus mengubah parameter fungsi search pada re.search menjadi **re.search("^From")**.

```

1  import re
2  handle=open('mbox-short.txt')
3  count = 0
4  for line in handle:
5      line=line.rstrip()
6      if re.search('^From:', line):
7          count += 1
8      print(line)
9      print("Count: ",count)

```

Kode di atas dapat digantikan juga dengan fungsi string.startswith("From :").

8.3.6 Meta Character, Escaped Character, Set of Character, dan Fungsi Regex pada Library Python

Sebelum menggunakan fungsi regex perlu diketahui terlebih dahulu meta character / special character dan kegunaannya pada pola regex seperti pada tabel 8.1

Tabel 8.1: Special Character pada Python

Karakter	Kegunaan	Contoh	Arti Contoh
[]	Kumpulan karakter	"[a-zA-Z]"	1 karakter antara a-z kecil atau A-Z besar
\{ }	Karakter dengan arti khusus dan escaped character	\{ }d	Angka / digit
.	Karakter apapun kecuali newline	say.n.	Tidak bisa diganti dengan karakter apapun, misal "sayang" akan valid
^	Diawali dengan	^From	Diawali dengan From
\$	Dakhiri dengan	this\$	Diakhiri dengan kata this
*	0 s/d tak terhingga karakter	\{ }d*	ada digit minimal 0 maksimal tak terhingga
?	ada atau tidak (opsional)	\{ }d?	Boleh ada atau tidak ada digit sebanyak
+	1 s/d tak terhingga karakter	\{ }d+	Minimal 1 s/d tak terhingga karakter
{ }	Tepat sebanyak yang ada para { }	\{ }d{2}	Ada tepat 2 digit
()	Pengelompokan karakter / pola	(sayalkamu)	saya atau kamu sebagai satu kesatuan
	atau	\{ }d\{ }s	1 digit atau 1 spasi

Tabel 8.2: Escaped Character pada Regex

Special Characters	Kegunaan	Contoh
\b	Digunakan untuk mengetahui apakah suatu pola berada di awal kata atau akhir kata	"R\bin" "Rain\b"
\d	Digunakan untuk mengetahui apakah karakter adalah sebuah digit (0 s/d 9)	\d
\D	Digunakan untuk mengetahui apakah karakter yang bukan digit	\D
\s	Digunakan untuk mengetahui apakah karakter adalah whitespace (spasi, tab, enter)	\s
\S	Digunakan untuk mengetahui apakah karakter adalah BUKAN whitespace (spasi, tab, enter)	\S
\w	Digunakan untuk mengetahui apakah karakter adalah word (a-z, A-Z, 0-9, dan _)	\w
\W	Digunakan untuk mengetahui apakah karakter adalah BUKAN word (a-z, A-Z, 0-9, dan _)	\W
\A	Digunakan untuk mengetahui apakah karakter adalah berada di bagian depan dari kalimat	"\AThe"
\Z	Digunakan untuk mengetahui apakah karakter adalah berada di bagian akhir dari kalimat	"End\Z"

Pada Python terdapat beberapa special character (escaped characters) seperti pada tabel 8.2

Pada Python terdapat beberapa penggunaan himpunan character dengan menggunakan simbol [], pada tabel 8.3

Tabel 8.3: Himpunan Karakter pada Regex

[abc]	Mencari pola 1 huruf a, atau b, atau c
[a-c]	Mencari pola 1 huruf a s/d c
[^bmx]	Mencari pola 1 huruf yang bukan b,m, atau x
[012]	Mencari pola 1 huruf 0, atau 1, atau 2
[0-3]	Mencari pola 1 huruf 0 s/d 3
[0-2][1-3]	Mencari pola 2 huruf: 01, 02, 03, 11, 12, 13, 21, 22, 23
[a-zA-Z]	Mencari pola 1 huruf a-Z

Pada Python terdapat 4 buah fungsi yang bisa dipakai untuk menggunakan Regex seperti pada tabel 8.4

Tabel 8.4: Fungsi Regex pada Python

Nama Fungsi	Kegunaan
findall	mengembalikan semua string yang sesuai pola (matches)
search	mengembalikan string yang sesuai pola (match)
split	memecah string sesuai pola
sub	mengganti string sesuai dengan pola yang cocok

8.4 Kegiatan Praktikum

Kasus 8.1 Buatlah program untuk menghitung berapa huruf hidup pada sebuah kalimat per kata

Kasus 8.2 Buatlah program untuk membuat mengubah susunan format tanggal dari YYYY-MM-DD menjadi DD-MM-YYYY

Kasus 8.3 Buatlah program untuk mengetahui apakah suatu kalimat adalah palindrom atau bukan! Palindrom adalah kalimat yang jika dibalik sama saja. Misalnya: Step on no pets, Pull up If I pull up, Some men interpret nine memos, dan Madam, In Eden I'm Adam.

Kasus 8.4 Buatlah program untuk mengambil beberapa kata dari suatu kalimat yang diinputkan. Misal kalimat: "saya akan pergi sekolah". Akan diambil 1 kata, maka hasilnya: "saya", "akan", "pergi", "sekolah". Sedangkan jika 2 kata maka hasilnya: "saya akan", "akan pergi", "pergi sekolah".

8.4.1 Pembahasan Soal 1

Untuk menghitung huruf hidup digunakan tahapan sebagai berikut:

- minta input kalimat yang akan dicari
- ubah string menjadi huruf kecil semua agar tidak membedakan huruf besar dan kecil
- siapkan variabel total jumlah huruf hidup aiueo dan set nilai awal = 0
- carilah ada berapa huruf 'a','i','u','e','o' pada kalimat yang diinputkan dengan method count()

Hasil:

```
1 a_string = "AnTonIus"
2 lowercase = a_string.lower()
3 total = 0
4 for x in "aiueo":
5     jml = lowercase.count(x)
6     total += jml
7 print(total)      #hasil = 4
```

8.4.2 Pembahasan Soal 2

Langkah yang harus dilakukan bisa 2 macam, yaitu dengan memecah / split string dan menyusunnya kembali sesuai yang diinginkan.

```
1 tgl = "2020-12-01"
2 hasil = tgl.split("-")
3 tgl2 = hasil[2]+"-"+hasil[1]+"-"+hasil[0]
4 print(tgl2)
```

8.4.3 Pembahasan Soal 3

Palindrom dapat diperiksa dengan membalik kata tersebut dan kemudian mencocokkannya dengan string aslinya. Jika sama maka palindrom, jika tidak maka bukan palindrom.

```
1 satu = "Step! on, no.. pets??"
2 satu = satu.lower()
3
4 satu = ''.join([i for i in satu if i.isalpha()])      #buang semua yang bukan alfabet
5
```

```

6 dua = satu[::-1]
7 if dua == satu:
8     print("palindrom")
9 else:
10    print("bukan")

```

8.4.4 Pembahasan Soal 4

Untuk mengambil beberapa kata dari kalimat, digunakan logika sebagai berikut:

- Ubah kalimat menjadi huruf kecil semua (lowercase)
- Pecah (split), kalimat dipisah berdasarkan spasi
- Untuk setiap kata yang sudah dipisah tersebut, maka ambil indeks kata per n buah, misal 1 buah jika n=1, 2 jika n=2. Indeks yang diambil adalah dimulai dari 0, jika n=2 maka ambil 0 dan 1, lalu ambil 1 dan 2, 2 dan 3, 3 dan 4, dst...dilakukan dalam perulangan.

```

1  #Kalimat
2  text = 'A quick brown fox jumps over the lazy dog.'
3
4  def ambil_kata_kalimat(kalimat, n):
5      kalimat = kalimat.lower() #ubah huruf kecil
6      hasil_akhir = [] #siapkan tempat hasil
7      hasil = kalimat.split() #pecah berdasarkan spasi
8      for i in range(0, len(hasil)): #loop per hasil pecah
9          tmp = ' '.join(hasil[i:i + n]) #ambil per indeks
10         hasil_akhir.append(tmp) #tambahkan ke list
11
12     return hasil_akhir #return
13
14 print(ambil_kata_kalimat(text, 2))

```

Hasil

```

['a quick', 'quick brown', 'brown fox', 'fox jumps',
'jumps over', 'over the', 'the lazy',
'lazy dog.', 'dog.']

```

8.5 Latihan Mandiri

Latihan 8.1 Buatlah sebuah program yang dapat mendeteksi apakah suatu kata adalah anagram dari kata lainnya atau bukan! Anagram adalah kata yang dibolak-balik susunan hurufnya sama. Misal: mata anagram dengan atma, maat, taam, tama, dsb. ■

Latihan 8.2 Buatlah suatu program yang dapat menghitung frekuensi kemunculan suatu kata yang ada pada String. Misal terdapat kalimat "Saya mau makan. Makan itu wajib. Mau siang atau malam saya wajib makan". Ditanyakan kata "makan". Output: makan ada 3 buah ■

Latihan 8.3 Buatlah suatu program yang dapat menghapus semua spasi yang berlebih pada sebuah string, dan menjadikannya satu spasi normal! Misal: "saya tidak suka memancing ikan " Output: "saya tidak suka memancing ikan" ■

Latihan 8.4 Buatlah suatu program mengetahui kata terpendek dan terpanjang dari suatu kalimat yang diinputkan! Misal: "red snakes and a black frog in the pool" Output: terpendek: a, terpanjang: snakes ■

8.5.1 Penggunaan findall

Kita akan mencoba penggunaan fungsi **findall** untuk mencari semua pola sebagai berikut:

```
1 import re
2
3 txt = "Sang mata-mata sedang memata-matai kasus kaca mata di toko Matahari"
4 x = re.findall("mata", txt)
5 y = re.findall("saya", txt)
6 for i in x:
7     print(i)
8
9 if (y):
10    print("Ada yang cocok!")
11 else:
12    print("Tidak ada yang cocok!")
```

Hasil:

```
mata #pada mata-mata
mata #pada mata-mata
mata #pada memata-matai
mata #pada memata-matai
mata #kaca mata
Tidak ada yang cocok #karena tidak ada 'saya'
```

Perhatikan bagian Matahari tidak muncul karena Matahari menggunakan huruf besar

Contoh lain fungsi **findall**:

```
1 import re
2 handle=open('mbox-short.txt')
3 for line in handle:
4     line=line.rstrip()
5     x=re.findall('\S+\S+', line)
6     if len(x)>0:
7         print(x)
```

Hasil:

```
['stephen.marquard@uct.ac.za']
['<postmaster@collab.sakaiproject.org>']
['<200801051412.m05ECIaH010327@nakamura.uits.iupui.edu>']
['<source@collab.sakaiproject.org>;']
['<source@collab.sakaiproject.org>;']
```

```
['<source@collab.sakaiproject.org>;']
['apache@localhost']
['source@collab.sakaiproject.org;']
['stephen.marquard@uct.ac.za']
['source@collab.sakaiproject.org']
....dst
```



Ada beberapa format email yang tidak sesuai format, seperti mengandung karakter <, sehingga kita perlu mengganti format regex nya menjadi: `[a-zA-Z0-9]*@[a-zA-Z]`. Silahkan ubah dibagian baris ke-5.

8.5.2 Penggunaan search

Kita akan mencoba penggunaan fungsi **search** untuk mencari pola sebagai berikut:

```
1 import re
2
3 txt = "Sang mata-mata sedang memata-matai kasus kaca mata di toko Matahari"
4 x = re.search("\s", txt)
5 y = re.search("saya", txt)
6
7 print("Spasi ditemukan di:", x.start())
8 print(y)
```

Hasil:

4

None

Contoh lain fungsi **search**: Pada mbox kita ingin menemukan kata-kata:

```
X-DSPAM-Confidence: 0.847
5X-DSPAM-Probability: 0.0000
X-DSPAM-Confidence: 0.6178
X-DSPAM-Probability: 0.0000
```

Untuk melakukannya dapat digunakan regex: `X-.*: [0-9.]+`

```
1 import re
2 handle=open('mbox-short.txt')
3 for line in handle:
4     line=line.rstrip()
5     if(re.search('^X-.*: [0-9.]+', line)):
6         print(line)
```

Hasil:

```
X-DSPAM-Confidence: 0.8475
X-DSPAM-Probability: 0.0000
X-DSPAM-Confidence: 0.6178
X-DSPAM-Probability: 0.0000
```

```
X-DSPAM-Confidence: 0.6961
X-DSPAM-Probability: 0.0000
X-DSPAM-Confidence: 0.7565
X-DSPAM-Probability: 0.0000
dst...
```

8.5.3 Penggunaan split

Kita akan mencoba penggunaan fungsi **split** untuk memecah string sebagai berikut:

```
1 import re
2
3 txt = "The rain in Spain"
4 x = re.split("\s", txt)
5 print(x)
6 y = re.split("\s", txt, 1) #split 1 kata pertama
7 print(x)
```

Hasil:

```
['The', 'rain', 'in', 'Spain']
['The', 'rain in Spain']
```

8.5.4 Penggunaan sub

Kita akan mencoba penggunaan fungsi **sub** untuk replace pola sebagai berikut:

```
1 import re
2
3 txt = "Sang mata-mata sedang memata-matai kasus kaca mata di toko Matahari"
4 x = re.sub("\s", "-", txt) #mengganti spasi dengan -
5 print(x)
6 y = re.sub("\s", "*", txt, 2) #mengganti spasi dengan * 2 saja
7 print(y)
```

Hasil:

```
Sang-mata-mata-sedang-memata-matai-kasus-kaca-mata-di toko-Matahari
Sang*mata-mata*sedang memata-matai kasus kaca mata di toko Matahari
```

Latihan 8.5 Anda diminta untuk mencari seluruh teks yang berupa tanggal dengan format YYYY-MM-DD dan kemudian seluruh tanggal tersebut diambil dan ditampilkan kembali dalam format DD-MM-YYYY ditambah dengan perhitungan selisih dengan tanggal sekarang dalam hari.

Contoh:

Pada tanggal 1945-08-17 Indonesia merdeka. Indonesia memiliki beberapa pahlawan nasional, seperti Pangeran Diponegoro (TL: 1785-11-11), Pattimura (TL: 1783-06-08) dan Ki Hajar Dewantara (1889-05-02).

Hasil:

```
1945-08-17 00:00:00 selisih 27209 hari
1785-11-11 00:00:00 selisih 85561 hari
1783-06-08 00:00:00 selisih 86448 hari
1889-05-02 00:00:00 selisih 47769 hari
```

Latihan 8.6 Anda diminta untuk mencari seluruh teks yang berupa email dan kemudian ambil semua username dari email tersebut untuk digenerate password random 8 karakter yang terdiri dari angka dan huruf.

Contoh:

Berikut adalah daftar email dan nama pengguna dari mailing list:

anton@mail.com dimiliki oleh antonius

budi@gmail.co.id dimiliki oleh budi anwari

slamet@getnada.com dimiliki oleh slamet slumut

matahari@tokopedia.com dimiliki oleh toko matahari

Hasil:

anton@mail.com username: anton , password: 8u78A2UD

budi@gmail.co.id username: budi , password: bdP066Ld

slamet@getnada.com username: slamet , password: Ab1FiHXb

matahari@tokopedia.com username: matahari , password: 5KYyaP6